Preentrega Módulo 2: Orquestación Multi-Agente

Proyecto: Gestor de Pedidos e Información para Comida Rápida (GustoBot)

Autor: Marcelo Troncoso **GitHub:** github.com/matronc/n8n_avanzado

Patrón: Manager-Worker | Plataforma: n8n Enterprise | Estatus: Validado & Operativo

1. VISIÓN GENERAL DEL ARQUITECTURA MANAGER-WORKER

Evolución de la arquitectura monolítica inicial hacia un organigrama digital modular de grado empresarial. El workflow Manager coordina la captura, filtrado, triaje con IA y delegación síncrona hacia Sub-workflows Workers especializados.

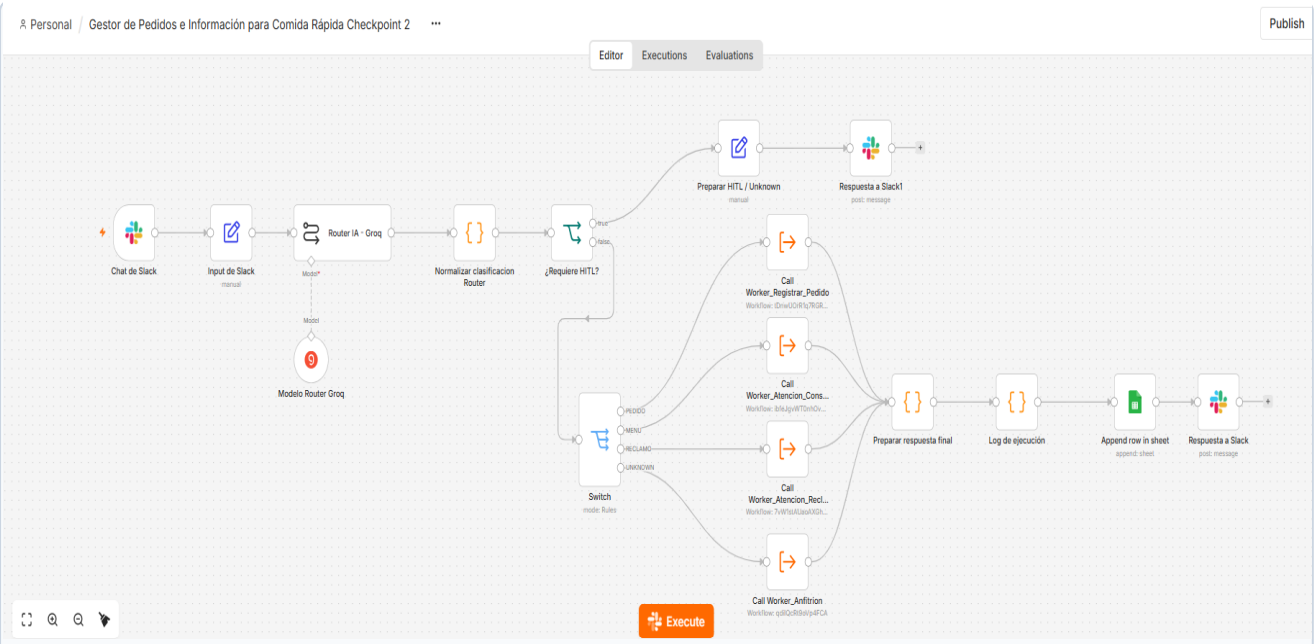


Figura 1.1: Visión global del workflow "Manager" en n8n con enrutamiento inteligente, módulo HITL y delegación a Workers.

1 Creación de Sub-workflows Worker

Implementación de Workers independientes en lienzos dedicados: `Worker_Registrar_Pedido`, `Worker_Atencion_Consultas`, `Worker_Atencion_Reclamos` y `Worker_Anfitrion`.

2 Trigger Nativo de Entrada

Configuración del nodo de disparo `Execute Workflow Trigger` en cada Worker para garantizar la recepción estructurada y segura de los datos transmitidos.

2. INGESTA Y MITIGACIÓN DE PAYLOAD ("PASILLO DE LA MUERTE")

Captura de eventos en tiempo real desde Slack y sanitización visual mediante nodos de edición para prevenir la saturación de memoria por metadatos masivos redundantes.

Trigger de Ingesta (Slack Webhook)

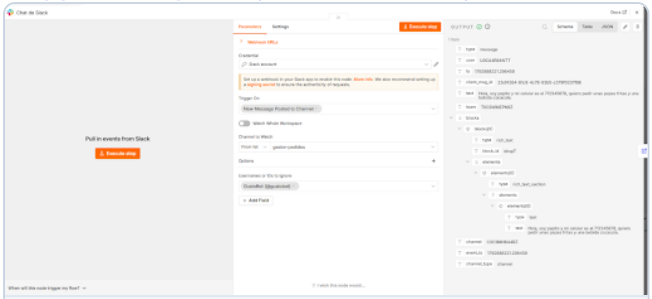


Figura 2.1: Disparo nativo por nuevo mensaje en el canal #gestor-pedidos.

Limpieza y Filtrado de Datos (Input)

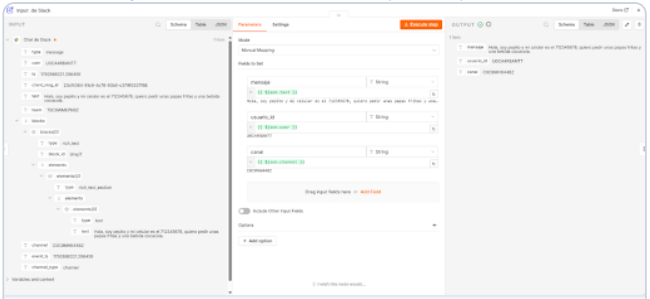


Figura 2.2: Extracción estricta del texto, ID de usuario y canal.

7 Mitigación del "Pasillo de la Muerte" de Datos

Mediante el nodo `Input de Slack` (Edit Data), se descartan metadatos pesados, objetos de bloques JSON extendidos y tokens del Webhook. Se normaliza la salida a una carga mínima viable (`mensaje` , `usuario_id` , `canal`), reduciendo drásticamente el consumo de memoria en la orquestación posterior.

Contrato de Entrada Sanitizado (JSON)

```
{
  "mensaje": "Hola, soy pepito y mi celular es el 712345678, quiero pedir unas papas fritas y una bebida cocacola.",
  "usuario_id": "U0CA4R8ANTT",
  "canal": "C0C9MH644BZ"
}
```

3. TRIAJE CON IA, TAXONOMÍA CERRADA Y ENRUTAMIENTO DETERMINISTA

Clasificación del lenguaje natural desestructurado mediante LLM (Groq) acotado a categorías de negocio estrictas, con filtro de seguridad y bifurcación Switch.

Router IA (Groq)

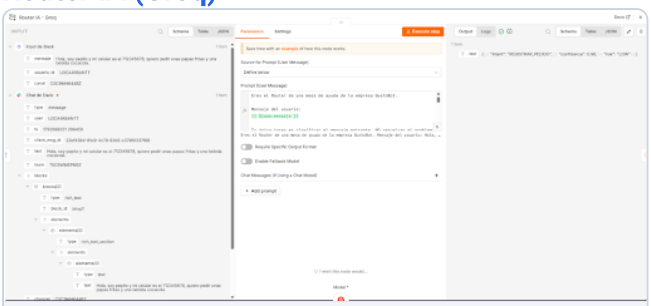


Figura 3.1: Prompt del sistema acotado a taxonomía cerrada.

Normalización Código JS

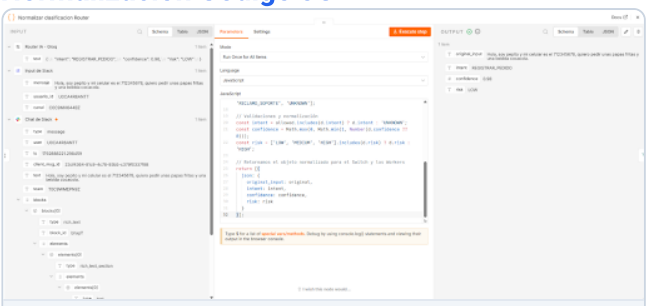


Figura 3.2: Sanitización de respuesta del Router IA.

8-9 Taxonomía Cerrada y Triaje

El Router IA asigna intenciones acotadas: `REGISTRAR_PEDIDO`, `CONSULTA_MENU`, `RECLAMO_SOPORTE` o `UNKNOWN`, asignando niveles de riesgo y confianza.

9 Escape HITL (Human-in-the-Loop)

Evaluación visual con el nodo `¿Requiere HITL?`: Si `risk == HIGH` o `confidence < 0.75`, redirige el caso a supervisión humana por Slack sin invocar Workers.

Validación de Escape HITL

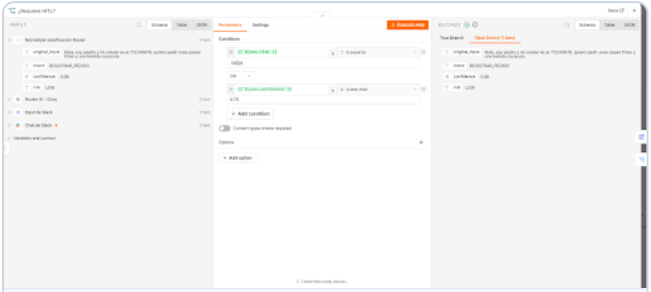


Figura 3.3: Filtro de seguridad (risk/confidence).

Bifurcación Determinista (Switch)

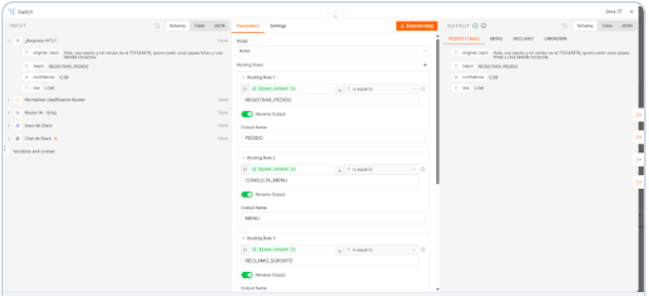


Figura 3.4: Reglas fijas costo cero tokens.

4. DELEGACIÓN, CONTRATOS DE SALIDA Y MANEJO DE ERRORES

Sincronización síncrona mediante sub-workflows, definición de respuesta JSON unificada y log de auditoría del sistema.

Delegación a Worker (Execute WF)

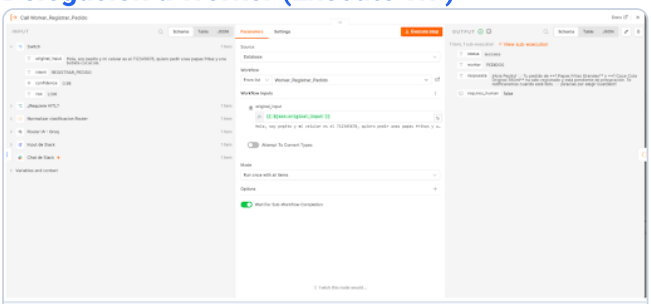


Figura 4.1: Opción Wait For Sub-Workflow activada.

Consolidación de Respuesta JSON

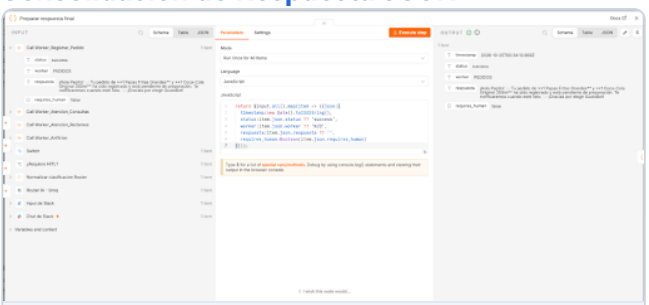


Figura 4.2: Estandarización de salida JSON limpia.

5-6 Sincronización y Pausa

Activación obligatoria de `Wait For Sub-Workflow Completion` en los nodos de delegación para pausar al Manager hasta obtener el retorno estructurado del Worker.

9,10 Resiliencia y Manejo de Errores

Configuración de salidas de fallo en Workers para retornar objetos de contingencia estandarizados (`{"status": "error", "message": "..."}`) evitando caídas catastróficas.

Persistencia y Trazabilidad (Log de Ejecución)

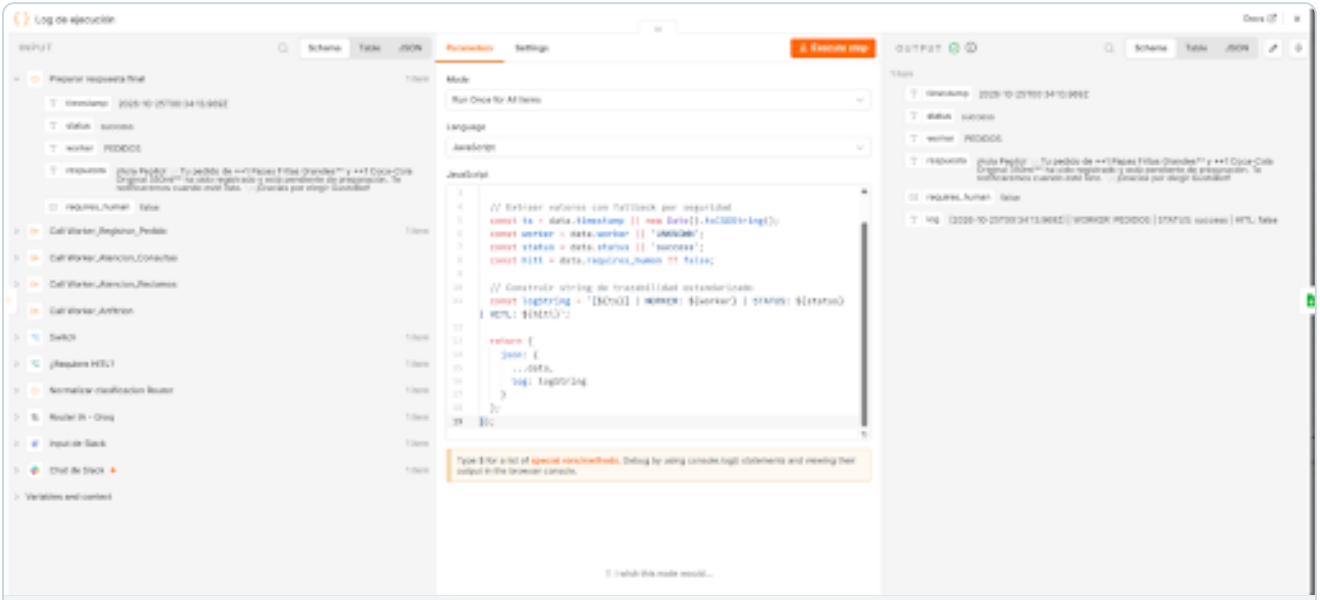


Figura 4.3: Registro auditable con marca de tiempo, worker invocado, estado y flag HITL.